

AeroCardia Normative Reference Spec v2.5

AeroCardia Normative Reference Engine

Complete Implementation Specification for Claude Code — v2.5

1. OVERVIEW

This document is a complete, self-contained specification for implementing the AeroCardia normative percentile engine. All normative parameters, algorithms, and design decisions are defined here. A small number of parameter values are flagged for verification against the cited source papers before finalising — these are marked explicitly with **Claude Code: verify** notes throughout the spec.

Three output category scores (never aggregated into one): - Pulmonary Score - Cardiovascular Score - Functional Mobility Score

User profile fields required at runtime (stored in database):

Field	Type	Notes
age	float	Years
sex	string	"male" or "female"
height_cm	float	Required for GLI spirometry and 6MWT
weight_kg	float	Required for 6MWT
ethnicity	string	Required for GLI spirometry. See Section 5.1.2 for valid values

Protocol versions: Three session lengths are supported (15-min, 20-min standard, 30-min). All versions produce the same three domain scores and feed the same normative engine. The number of input metrics differs between versions; the scoring functions normalise weights automatically over whichever metrics were collected. See Section 1.5.

Design rule: Always store raw test scores in the database. Never store computed percentiles. This allows recomputation when normative references are updated.

1.5 PROTOCOL VERSIONS

Three session lengths are defined. Version selection is driven by available user time, not by age or condition (those drive test *variant* selection within a version).

Version	Duration	Tests administered
15-min	~15 min	Resting Baseline (3 min), Spirometry (2 trials), YMCA/6MWT, Balance (age-routed), STS/Squat (age-routed)
20-min (Standard)	~20 min	As above, plus: Resting Baseline (4 min, adds HRV), Breath-Hold, Spirometry (3 trials)
30-min	~30 min	All 20-min tests, plus: Front Plank, Wall Sit

Key differences by version:

- **Resting Baseline:** 15-min = 3 min quiet breathing only (captures resting HR, SpO₂, I:E ratio). 20/30-min = 4 min including 1 min controlled deep breathing (additionally captures HRV/RMSSD).
- **Spirometry:** 15-min = 2 trials. 20/30-min = 3 trials.
- **Breath-Hold:** Absent from 15-min version (contributes only 5% of Pulmonary Score; thinnest normative base in the battery). Present in 20/30-min versions.
- **Front Plank & Wall Sit:** 30-min only.

Scoring under version-conditional metrics: `pulmonary_score()` and `mobility_score()` accept optional parameters for metrics not collected in all versions. Missing metrics are excluded and remaining weights are renormalised to sum to 1.0. See Section 3.

1.6 PRE-TEST REQUIREMENTS

Apply to all three versions before any test begins.

- User must not have eaten a large meal within 2 hours.
- No vigorous exercise within 2 hours.
- No caffeine within 1 hour (affects resting HR and HRV).
- User should be seated and at rest for at least 5 minutes before the session begins.
- Room temperature should be comfortable (extreme heat/cold affects spirometry and HR).
- The app should confirm these requirements via a checklist before unlocking the Start button.

2. MODULE ARCHITECTURE

2.1 Entry Point

```
def compute_percentile(metric_id: str, raw_score: float, age: float,
                      sex: str = None, height_cm: float = None,
                      weight_kg: float = None,
                      ethnicity: str = "caucasian") -> float:
    """
    Returns percentile 0-100. For inverted metrics (lower = better),
```

already returns 100 - raw so that higher is always better.
ethnicity: required for fev1/fvc/fev1_fvc – pass from user profile.
weight_kg: required for sixmwt_distance – pass from user profile.
 """

2.2 Metric IDs

metric_id	Description	Inverted?	Extra args needed	Routing condition
resting_hr	Resting heart rate (bpm)	YES	age	All users
breath_hold	Max breath-hold (seconds)	No	—	20/30-min versions only
fev1	FEV1 (litres)	No	age, sex, height_cm, ethnicity	All users
fvc	FVC (litres)	No	age, sex, height_cm, ethnicity	All users
fev1_fvc	FEV1/FVC ratio	No	age, sex, height_cm, ethnicity	All users
ymca_recovery_hr	1-min post-step recovery HR (bpm)	YES	age	No cardiac condition
sixmwt_distance	6-min walk distance (metres)	No	age, sex, height_cm, weight_kg	Cardiac condition
balance_standard	Avg single-leg balance eyes-open (s, avg R+L, ceil 60)	No	age	Age > 45, OR any mobility-limiting, vestibular, or neurological condition
balance_young	Eyes-open + eyes-closed balance composite score (0–100). raw_score = eyes_open_avg_s + eyes_closed_avg_s (sum, not average; max 120)	No	—	Age ≤ 45, no mobility-limiting, vestibular, or neurological condition
sts_30	30-second sit-to-stand (reps)	No	age	Age > 45 or mobility-limiting condition
squat_exhaustion	Bodyweight squats to exhaustion, 2-min cap (reps)	No	sex	Age ≤ 45, no mobility-limiting condition
hrv_rmssd	Resting HRV — RMSSD (ms)	No	age, sex	20/30-min versions only
plank_duration		No	age, sex	30-min version only

metric_id	Description	Inverted?	Extra args needed	Routing condition
	Front plank hold time (seconds, 3-min cap)			
wall_sit_duration	Wall sit hold time (seconds, 2-min cap)	No	age, sex	30-min version only

I:E ratio is captured during resting baseline but is a **clinical flag only** (not a scored percentile metric). See Section 5.8. It does not appear in `compute_percentile`.

SpO₂ is also captured during resting baseline but does **not** appear in `compute_percentile`. It is passed directly to `pulmonary_score()` as the `spo2_ok: bool` parameter (via `spo2_flag()` in `normative/spo2.py`). See Section 5.7.

Routing logic: The app determines which `metric_id` to use based on the user's age and condition list stored in their profile. Claude Code should refer to the conditions schema to determine which conditions qualify as cardiac-limiting or mobility-limiting. The normative engine itself is stateless — routing happens in the application layer before `compute_percentile` is called.

Note on balance vs. STS routing asymmetry: `balance_standard` routing includes vestibular and neurological conditions (in addition to mobility-limiting conditions) because those directly impair postural stability. `sts_30` routing uses only mobility-limiting conditions — vestibular/neurological conditions do not necessarily prevent squat or sit-to-stand testing. This asymmetry is intentional.

3. COMPOSITE SCORING

3.1 Pulmonary Score

Weights are defined for the full 20/30-min version. When a metric is absent (e.g. `breath_hold` in the 15-min version, `hrv_rmssd` in the 15-min version), it is excluded and remaining weights are renormalised so they always sum to 1.0.

Full-version base weights:

Metric	Weight	Notes
resting_hr	0.15	Inverted
hrv_rmssd	0.10	20/30-min only
breath_hold	0.05	20/30-min only; preliminary, down-weighted
fev1	0.20	
fvc	0.20	
fev1_fvc	0.30	Highest weight — most sensitive spirometry marker

```
# normative/scoring.py
def pulmonary_score(resting_hr_pct: float,
                    fev1_pct: float, fvc_pct: float, fev1_fvc_pct: float,
```

```

        spo2_ok: bool,
        hrv_rmssd_pct: float = None,
        breath_hold_pct: float = None) -> float:
    """
    hrv_rmssd_pct and breath_hold_pct are optional – pass None when not collected
    (i.e. 15-min version). Weights renormalise automatically over present metrics.
    """
    components = [
        (resting_hr_pct, 0.15),
        (fev1_pct, 0.20),
        (fvc_pct, 0.20),
        (fev1_fvc_pct, 0.30),
    ]
    if hrv_rmssd_pct is not None:
        components.append((hrv_rmssd_pct, 0.10))
    if breath_hold_pct is not None:
        components.append((breath_hold_pct, 0.05))

    total_weight = sum(w for _, w in components)
    score = sum(v * w for v, w in components) / total_weight

    if not spo2_ok:          # SpO2 < 95%: binary cap, not a weighted input
        score = min(score, 40.0)
    return round(score, 1)

```

3.2 Cardiovascular Score

```

# normative/scoring.py (continued)
def cardiovascular_score(cardio_pct: float) -> float:
    """
    cardio_pct is either:
    - ymca_recovery_hr percentile (standard users)
    - sixmwt_distance percentile (cardiac-condition users)
    Both are already 0-100 higher-is-better. Caller passes the appropriate one.
    VO2max (from YMCA only) is a secondary display-only output – not part of score.
    """
    return round(cardio_pct, 1)

```

UI note for cardiac-condition users: Display label as “Functional Exercise Capacity” rather than “Cardiovascular Fitness” and add disclaimer: “Score reflects 6-minute walk distance relative to predicted values for your age, sex, height, and weight. This measures functional walking capacity rather than peak cardiovascular fitness.”

3.3 Functional Mobility Score

Plank and wall sit are only collected in the 30-min version. When absent, weights renormalise to balance (0.50) and STS (0.50), preserving longitudinal comparability of the base components across versions.

Full-version base weights (30-min):

Metric	Weight
balance	0.35

Metric	Weight
sts / squat	0.35
plank	0.15
wall_sit	0.15

```
# normative/scoring.py (continued)
def mobility_score(balance_pct: float, sts_pct: float,
                  plank_pct: float = None,
                  wall_sit_pct: float = None) -> float:
    """
    balance_pct: from balance_standard (population percentile 0-100)
                  OR balance_young (normalised score 0-100, see note below)
    sts_pct:      from sts_30 OR squat_exhaustion (both population percentiles)
    plank_pct:    from plank_duration - 30-min version only, else None
    wall_sit_pct: from wall_sit_duration - 30-min version only, else None
    Weights renormalise automatically over present metrics.

    ⚠️ SCALE NOTE: balance_young produces a normalised raw score (0-100),
    not a population percentile. It is intentionally treated as equivalent
    here because no adequate reference population exists for the composite
    eyes-open + eyes-closed protocol in young adults. The 0-100 scale is
    designed to be interpretively comparable (50 = average, 100 = perfect),
    but the value is not strictly a percentile rank. UI should note this
    distinction when displaying the Functional Mobility Score for users who
    took the young balance protocol (see Section 5.4.2).
    """
    components = [
        (balance_pct, 0.35),
        (sts_pct, 0.35),
    ]
    if plank_pct is not None:
        components.append((plank_pct, 0.15))
    if wall_sit_pct is not None:
        components.append((wall_sit_pct, 0.15))

    total_weight = sum(w for _, w in components)
    return round(sum(v * w for v, w in components) / total_weight, 1)
```

4. AGE BUCKET HELPERS

Define these once; all metric modules import them.

```
# normative/utils.py
def _bucket(age, buckets):
    """
    Returns the age bucket tuple (lo, hi) that contains age.
    Clamps to buckets[0] if age is below the lowest bucket (e.g. age < 18),
    and to buckets[-1] if age is above the highest bucket.
    The engine assumes the app has validated age >= 18; this clamp is a
    safety fallback only.
```

```

    """
    for lo, hi in buckets:
        if lo <= age <= hi:
            return (lo, hi)
    if age < buckets[0][0]:
        return buckets[0] # below minimum - clamp to youngest bucket
    return buckets[-1] # above maximum - clamp to oldest bucket

def nhanes_age_bucket(age):
    return _bucket(age, [(18, 29), (30, 39), (40, 49), (50, 59), (60, 69), (70, 95)])

def ymca_age_bucket(age):
    return _bucket(age, [(18, 25), (26, 35), (36, 45), (46, 55), (56, 65), (66, 99)])

def balance_age_bucket(age):
    return _bucket(age, [(18, 39), (40, 49), (50, 59), (60, 69), (70, 79), (80, 99)])

def sts_age_bucket(age):
    return _bucket(age, [(18, 35), (36, 45), (46, 55), (56, 59),
                          (60, 64), (65, 69), (70, 74), (75, 79), (80, 99)])

def hrv_age_bucket(age):
    return _bucket(age, [(18, 29), (30, 39), (40, 49), (50, 59), (60, 69), (70, 99)])

def plank_age_bucket(age):
    return _bucket(age, [(18, 39), (40, 59), (60, 99)])

def wall_sit_age_bucket(age):
    return _bucket(age, [(18, 39), (40, 59), (60, 99)])

```

5. NORMATIVE REFERENCE IMPLEMENTATIONS

5.1 GLI-2012 Spirometry with Ethnicity (FEV1, FVC, FEV1/FVC)

Method: Pre-computed lookup table with bilinear interpolation on age and height. **Source:** ERS GLI API (gli-calculator.ersnet.org), called offline to populate `normative/data/gli_lookup.csv`. The API implements Quanjer PH et al. Eur Respir J 2012;40(6):1324–1343 (doi:10.1183/09031936.00080312).

The lookup table stores L, M, S for each (sex, age, height_cm, ethnicity) grid point. At runtime, L, M, S are retrieved by bilinear interpolation and fed into the standard GLI Z-score formula to compute the percentile.

5.1.1 Lookup Table Format

File: `normative/data/gli_lookup.csv`

Relevant columns (others from the API response can be ignored):

Column	Type	Description
sex	string	"m" or "f"
age	int	Age in years
height_cm	float	Height in cm

Column	Type	Description
ethnicity	int	1=Caucasian, 2=African American, 3=NE Asian, 4=SE Asian, 5=Other
fev1L, fev1M, fev1S	float	GLI L, M, S for FEV1
fvcl, fvcM, fvcS	float	GLI L, M, S for FVC
fev1fvcl, fev1fvcM, fev1fvcS	float	GLI L, M, S for FEV1/FVC

Rows where L/M/S are null (unfilled API calls) are dropped at load time.

Current grid coverage (adults): - Ages: 19, 24, 30, 35, 41, 52, 62, 73, 84 (will expand as API quota allows) - Heights: 144.8–188.0 cm in ~2.5 cm increments (16 values) - Sex: m, f - Ethnicity: 1–5

Clamping: If a user's age or height falls outside the grid, clamp to the nearest grid boundary before interpolation. Document the clamped ranges in a module constant so they update automatically as the grid expands.

5.1.2 Ethnicity Mapping

User profiles store ethnicity as a string. Map to the integer code used in the CSV:

```
# normative/gli.py
ETHNICITY_CODE = {
    "caucasian": 1,
    "african_american": 2,
    "ne_asian": 3,
    "se_asian": 4,
    "other": 5,
}
# Any unknown string → 5 (Other)
```

5.1.3 Module Loading

```
# normative/gli.py – loaded on first import
import numpy as np
import pandas as pd
from scipy.stats import norm as scipy_norm

_df = pd.read_csv("normative/data/gli_lookup.csv")
_df = _df[_df["fev1M"].notna()].copy() # drop unfilled rows
_df["sex"] = _df["sex"].str.lower() # normalise to lowercase

# Pre-index by (sex, ethnicity) for fast lookup
GLI_LOOKUP = {
    (sex, eth): grp.sort_values(["age", "height_cm"]).reset_index(drop=True)
    for (sex, eth), grp in _df.groupby(["sex", "ethnicity"])
}

# Grid bounds – update automatically as CSV grows
GLI_AGE_BOUNDS = {k: (grp["age"].min(), grp["age"].max()) for k, grp in
GLI_LOOKUP.items()}
```



```

        GLI_HEIGHT_BOUNDS = {k: (grp["height_cm"].min(), grp["height_cm"].max()) for k, grp in
        GLI_LOOKUP.items()}

```

5.1.4 Bilinear Interpolation

```

# normative/gli.py (continued)
def _bilinear_lms(sex_code: str, eth_code: int,
                  age: float, height_cm: float, param: str):
    """
    Returns (L, M, S) for the given param by bilinear interpolation
    on the (age, height_cm) grid.
    param: 'fev1', 'fvc', or 'fev1fvc'
    """
    key = (sex_code, eth_code)
    grp = GLI_LOOKUP[key]

    # Clamp to grid bounds
    age_min, age_max = GLI_AGE_BOUNDS[key]
    h_min, h_max = GLI_HEIGHT_BOUNDS[key]
    age = float(np.clip(age, age_min, age_max))
    height_cm = float(np.clip(height_cm, h_min, h_max))

    ages = np.sort(grp["age"].unique())
    heights = np.sort(grp["height_cm"].unique())

    # Find bracketing indices
    ai = np.searchsorted(ages, age, side="right") - 1
    hi = np.searchsorted(heights, height_cm, side="right") - 1
    ai = int(np.clip(ai, 0, len(ages) - 2))
    hi = int(np.clip(hi, 0, len(heights) - 2))

    a0, a1 = ages[ai], ages[ai + 1]
    h0, h1 = heights[hi], heights[hi + 1]

    ta = (age - a0) / (a1 - a0)
    th = (height_cm - h0) / (h1 - h0)

    def _get(a, h, col):
        row = grp[(grp["age"] == a) & (grp["height_cm"] == h)]
        return float(row[col].iloc[0])

    def _interp(col):
        v00 = _get(a0, h0, col)
        v10 = _get(a1, h0, col)
        v01 = _get(a0, h1, col)
        v11 = _get(a1, h1, col)
        return (v00 * (1-ta) * (1-th) +
                v10 * ta * (1-th) +
                v01 * (1-ta) * th +
                v11 * ta * th)

    L = _interp(f"{param}L")
    M = _interp(f"{param}M")
    S = _interp(f"{param}S")
    return L, M, S

```

5.1.5 Core Percentile Function

```
# normative/gli.py (continued)
def gli_percentile(measured: float, age: float, sex: str,
                  height_cm: float, param: str,
                  ethnicity: str = "caucasian") -> float:
    """
    param:      'fev1', 'fvc', or 'fev1fvc'
    ethnicity:  'caucasian', 'african_american', 'ne_asian', 'se_asian', 'other'
    sex:        'male' or 'female'
    """
    sex_code = "m" if sex == "male" else "f"
    eth_code = ETHNICITY_CODE.get(ethnicity, 5)
    L, M, S = _bilinear_lms(sex_code, eth_code, age, height_cm, param)

    if abs(L) < 1e-10:
        z = np.log(measured / M) / S
    else:
        z = ((measured / M) ** L - 1.0) / (L * S)
    return float(scipy_norm.cdf(z) * 100)
```

5.1.6 Dispatch param names

The lookup CSV uses lowercase param prefixes (fev1 , fvc , fev1fvc). The compute_percentile dispatcher passes metric_id strings directly, so the mapping is:

metric_id	param arg to gli_percentile
fev1	"fev1"
fvc	"fvc"
fev1_fvc	"fev1fvc"

5.1.7 Validation

```
# Spot-check: male, Caucasian, age 30, height 172.7 cm
# Expected fev1M ~4.208 (matches raw CSV value – no interpolation needed)
L, M, S = _bilinear_lms("m", 1, 30, 172.7, "fev1")
assert abs(M - 4.208) < 0.01, f"fev1M mismatch: {M}"

# Interpolation check: age between grid points
L2, M2, S2 = _bilinear_lms("m", 1, 27, 172.7, "fev1")
# FEV1 predicted value declines with age after ~25 in males
# so M(age=24) > M(age=27) > M(age=30)
L0, M0, _ = _bilinear_lms("m", 1, 24, 172.7, "fev1")
assert M0 > M2 > M, f"Monotonicity check failed: {M0} > {M2} > {M}"

# Full percentile smoke test
pct = gli_percentile(4.0, 30, "male", 172.7, "fev1", "caucasian")
print(f"pct: {pct:.1f}") # expected ~45-55th percentile
```

5.2 NHANES Resting Heart Rate KDE

Method: Gaussian KDE fit offline; CDF lookup at runtime. Per decade age bucket. **Source:** NHANES Continuous Survey 2003-2018, CDC/NCHS **Inverted:** lower HR = better fitness → final percentile = 100 - kde_cdf(hr)

5.2.1 Download URLs

BPX examination files (variable: `BPXPPLS` — 60-sec pulse rate):

```
2003-2004: https://wwwn.cdc.gov/Nchs/Data/Nhanes/Public/2003/DataFiles/BPX_C.XPT
2005-2006: https://wwwn.cdc.gov/Nchs/Data/Nhanes/Public/2005/DataFiles/BPX_D.XPT
2007-2008: https://wwwn.cdc.gov/Nchs/Data/Nhanes/Public/2007/DataFiles/BPX_E.XPT
2009-2010: https://wwwn.cdc.gov/Nchs/Data/Nhanes/Public/2009/DataFiles/BPX_F.XPT
2011-2012: https://wwwn.cdc.gov/Nchs/Data/Nhanes/Public/2011/DataFiles/BPX_G.XPT
2013-2014: https://wwwn.cdc.gov/Nchs/Data/Nhanes/Public/2013/DataFiles/BPX_H.XPT
2015-2016: https://wwwn.cdc.gov/Nchs/Data/Nhanes/Public/2015/DataFiles/BPX_I.XPT
2017-2018: https://wwwn.cdc.gov/Nchs/Data/Nhanes/Public/2017/DataFiles/BPX_J.XPT
```

DEMO files (variable: `RIDAGEYR` — age in years):

```
2003-2004: https://wwwn.cdc.gov/Nchs/Data/Nhanes/Public/2003/DataFiles/DEMO_C.XPT
2005-2006: https://wwwn.cdc.gov/Nchs/Data/Nhanes/Public/2005/DataFiles/DEMO_D.XPT
2007-2008: https://wwwn.cdc.gov/Nchs/Data/Nhanes/Public/2007/DataFiles/DEMO_E.XPT
2009-2010: https://wwwn.cdc.gov/Nchs/Data/Nhanes/Public/2009/DataFiles/DEMO_F.XPT
2011-2012: https://wwwn.cdc.gov/Nchs/Data/Nhanes/Public/2011/DataFiles/DEMO_G.XPT
2013-2014: https://wwwn.cdc.gov/Nchs/Data/Nhanes/Public/2013/DataFiles/DEMO_H.XPT
2015-2016: https://wwwn.cdc.gov/Nchs/Data/Nhanes/Public/2015/DataFiles/DEMO_I.XPT
2017-2018: https://wwwn.cdc.gov/Nchs/Data/Nhanes/Public/2017/DataFiles/DEMO_J.XPT
```

MCQ files (medical conditions — CVD exclusion):

```
2003-2004: https://wwwn.cdc.gov/Nchs/Data/Nhanes/Public/2003/DataFiles/MCQ_C.XPT
2005-2006: https://wwwn.cdc.gov/Nchs/Data/Nhanes/Public/2005/DataFiles/MCQ_D.XPT
2007-2008: https://wwwn.cdc.gov/Nchs/Data/Nhanes/Public/2007/DataFiles/MCQ_E.XPT
2009-2010: https://wwwn.cdc.gov/Nchs/Data/Nhanes/Public/2009/DataFiles/MCQ_F.XPT
2011-2012: https://wwwn.cdc.gov/Nchs/Data/Nhanes/Public/2011/DataFiles/MCQ_G.XPT
2013-2014: https://wwwn.cdc.gov/Nchs/Data/Nhanes/Public/2013/DataFiles/MCQ_H.XPT
2015-2016: https://wwwn.cdc.gov/Nchs/Data/Nhanes/Public/2015/DataFiles/MCQ_I.XPT
2017-2018: https://wwwn.cdc.gov/Nchs/Data/Nhanes/Public/2017/DataFiles/MCQ_J.XPT
```

BPQ files (blood pressure questionnaire — medication exclusion):

```
2003-2004: https://wwwn.cdc.gov/Nchs/Data/Nhanes/Public/2003/DataFiles/BPQ_C.XPT
2005-2006: https://wwwn.cdc.gov/Nchs/Data/Nhanes/Public/2005/DataFiles/BPQ_D.XPT
2007-2008: https://wwwn.cdc.gov/Nchs/Data/Nhanes/Public/2007/DataFiles/BPQ_E.XPT
2009-2010: https://wwwn.cdc.gov/Nchs/Data/Nhanes/Public/2009/DataFiles/BPQ_F.XPT
2011-2012: https://wwwn.cdc.gov/Nchs/Data/Nhanes/Public/2011/DataFiles/BPQ_G.XPT
2013-2014: https://wwwn.cdc.gov/Nchs/Data/Nhanes/Public/2013/DataFiles/BPQ_H.XPT
2015-2016: https://wwwn.cdc.gov/Nchs/Data/Nhanes/Public/2015/DataFiles/BPQ_I.XPT
2017-2018: https://wwwn.cdc.gov/Nchs/Data/Nhanes/Public/2017/DataFiles/BPQ_J.XPT
```

5.2.2 Variable Reference

Variable	File	Description	Filter action
SEQN	all	Respondent ID	Join key across all files
BPXPLS	BPX	60-sec pulse rate (bpm)	Target variable
RIDAGEYR	DEMO	Age in years	Keep >= 18
MCQ160B	MCQ	Congestive heart failure	Exclude if == 1
MCQ160C	MCQ	Coronary heart disease	Exclude if == 1
MCQ160E	MCQ	Heart attack	Exclude if == 1
MCQ160F	MCQ	Stroke	Exclude if == 1
MCQ170L	MCQ	Currently has thyroid problem	Exclude if == 1 (use if present)
MCQ160M	MCQ	Ever had thyroid problem	Exclude if == 1 (fallback if MCQ170L absent)
BPQ050A	BPQ	Currently on prescribed BP medication	Exclude if == 1

Note: MCQ variable names are consistent across 2003-2018. If a variable is missing in a specific cycle, skip that filter for that cycle rather than raising an error.

5.2.3 KDE Fitting Script (Run Offline Once)

```
# normative/nhanes.py
import pandas as pd
import numpy as np
import pickle
from scipy.stats import gaussian_kde

NHANES_CYCLES = [
    ("BPX_C.XPT", "DEMO_C.XPT", "MCQ_C.XPT", "BPQ_C.XPT"),
    ("BPX_D.XPT", "DEMO_D.XPT", "MCQ_D.XPT", "BPQ_D.XPT"),
    ("BPX_E.XPT", "DEMO_E.XPT", "MCQ_E.XPT", "BPQ_E.XPT"),
    ("BPX_F.XPT", "DEMO_F.XPT", "MCQ_F.XPT", "BPQ_F.XPT"),
    ("BPX_G.XPT", "DEMO_G.XPT", "MCQ_G.XPT", "BPQ_G.XPT"),
    ("BPX_H.XPT", "DEMO_H.XPT", "MCQ_H.XPT", "BPQ_H.XPT"),
    ("BPX_I.XPT", "DEMO_I.XPT", "MCQ_I.XPT", "BPQ_I.XPT"),
    ("BPX_J.XPT", "DEMO_J.XPT", "MCQ_J.XPT", "BPQ_J.XPT"),
]

def build_nhanes_kde(raw_dir="nhanes_raw/", out="normative/data/nhanes_hr_kde.pkl"):
    frames = []
    for bpx_f, demo_f, mcq_f, bpq_f in NHANES_CYCLES:
        bpx = pd.read_sas(raw_dir + bpx_f, encoding="utf-8")
        demo = pd.read_sas(raw_dir + demo_f, encoding="utf-8")
        mcq = pd.read_sas(raw_dir + mcq_f, encoding="utf-8")
        bpq = pd.read_sas(raw_dir + bpq_f, encoding="utf-8")
        # XPT files store variable names as uppercase – normalize all to uppercase
```

```

for frame in [bpx, demo, mcq, bpq]:
    frame.columns = frame.columns.str.upper()
mcq_cols = ["SEQN"] + [c for c in
    ["MCQ160B", "MCQ160C", "MCQ160E", "MCQ160F", "MCQ160M", "MCQ170L"]
    if c in mcq.columns]
df = (bpx[["SEQN", "BPXPLS"]].
    .merge(demo[["SEQN", "RIDAGEYR"]], on="SEQN", how="inner")
    .merge(mcq[mcq_cols], on="SEQN", how="left")
    .merge(bpq[["SEQN", "BPQ050A"]], on="SEQN", how="left"))
frames.append(df)

df = pd.concat(frames, ignore_index=True)

# Filters
df = df[df["RIDAGEYR"] >= 18]
df = df[df["BPXPLS"].notna() & (df["BPXPLS"] > 0) & (df["BPXPLS"] < 200)]
for col in ["MCQ160B", "MCQ160C", "MCQ160E", "MCQ160F"]:
    if col in df.columns:
        df = df[df[col] != 1]
# ⚠ IMPORTANT – thyroid filter should be applied PER CYCLE, not post-concat.
# After concat, MCQ170L exists as a column (NaN for cycles that lacked it).
# The if/elif below always takes the MCQ170L branch, so MCQ160M never fires.
# Participants with thyroid conditions from cycles where only MCQ160M was
# available will leak through the filter. Claude Code should move this logic
# inside the per-cycle loop, before frames.append(df), checking each cycle's
# mcq columns individually.
if "MCQ170L" in df.columns:
    df = df[df["MCQ170L"] != 1]
elif "MCQ160M" in df.columns:
    df = df[df["MCQ160M"] != 1]
if "BPQ050A" in df.columns:
    df = df[df["BPQ050A"] != 1]

kde_dict = {}
for lo, hi in [(18, 29), (30, 39), (40, 49), (50, 59), (60, 69), (70, 95)]:
    data = df[(df["RIDAGEYR"] >= lo) & (df["RIDAGEYR"] <= hi)]["BPXPLS"].values
    assert len(data) >= 500, f"Too few samples: age {lo}-{hi}, n={len(data)}"
    kde_dict[(lo, hi)] = gaussian_kde(data, bw_method="silverman")
    print(f"Age {lo}-{hi}: n={len(data)}, median={np.median(data):.1f} bpm")

with open(out, "wb") as f:
    pickle.dump(kde_dict, f)
print(f"Saved to {out} – commit this file to the repo.")

```

Expected console output (if any bucket n < 500, debug exclusion logic):

```

Age 18-29: n=~2200, median HR=~70 bpm
Age 30-39: n=~2100, median HR=~70 bpm
Age 40-49: n=~2300, median HR=~71 bpm
Age 50-59: n=~2200, median HR=~72 bpm
Age 60-69: n=~2000, median HR=~72 bpm
Age 70-95: n=~1500, median HR=~73 bpm

```

5.2.4 Runtime Percentile Lookup

KDE_DICT must be a module-level global in `normative/nhanes.py` (not `__init__.py`), so `resting_hr_percentile` can reference it without a circular import. Load it at the top of `nhanes.py`, below the `build_nhanes_kde` function:

```
# normative/nhanes.py – module-level (loaded on first import, after build_nhanes_kde definition)

import pickle, numpy as np
from normative.utils import nhanes_age_bucket

with open("normative/data/nhanes_hr_kde.pkl", "rb") as f:
    KDE_DICT = pickle.load(f)

def resting_hr_percentile(hr_bpm: float, age: float) -> float:
    kde = KDE_DICT[nhanes_age_bucket(age)]
    raw_pct = kde.integrate_box_1d(-np.inf, hr_bpm) * 100
    return round(100.0 - raw_pct, 1) # invert: lower HR = higher fitness
```

5.3 YMCA 3-Minute Step Test — Recovery HR

Method: Normal distribution, parameters back-calculated from YMCA category boundaries **Source:** Golding LA et al. Y's Way to Physical Fitness. YMCA of USA, 1989 (3rd ed.) **Inverted:** lower recovery HR = better → percentile = 100 - norm.cdf(hr)

Protocol: 1. Stand in front of the step box and wait for the app to begin the cadence. 2. Step up and down on the box at the app-set cadence of **96 bpm** (24 complete cycles/min — up-up-down-down). Maintain full upright posture at the top of each step. Step box height: **30.5 cm (12 inches)** per YMCA standard. 3. Continue for exactly 3 minutes. Stop immediately if dizzy, overly breathless, or unsteady. 4. Sit down immediately when the 3-minute timer ends. Remain seated and still. 5. The app records 1-minute recovery HR automatically. Remain seated for 2 minutes before the next test.

Not cardiac-condition routed: Cardiac-condition users skip this test entirely and perform the 6MWT instead (Section 5.3b).

Age Bucket	Mean (bpm)	SD (bpm)
18-25	103	12
26-35	104	13
36-45	105	13
46-55	107	13
56-65	109	14
66+	111	14

```
# normative/ymca.py
from scipy.stats import norm
from normative.utils import ymca_age_bucket
```

```

YMCA_PARAMS = {
    (18, 25):(103, 12), (26, 35):(104, 13), (36, 45):(105, 13),
    (46, 55):(107, 13), (56, 65):(109, 14), (66, 99):(111, 14),
}

def ymca_hr_percentile(recovery_hr: float, age: float) -> float:
    mean, sd = YMCA_PARAMS[ymca_age_bucket(age)]
    return round(100.0 - norm.cdf(recovery_hr, loc=mean, scale=sd) * 100, 1)

def estimate_vo2max(recovery_hr: float, sex: str) -> tuple:
    """
    Display only – not scored. SEE ~6 mL/kg/min. Show as range.
    Returns: (point_estimate, low, high) – all in mL/kg/min.
    point_estimate: central regression value
    low: point_estimate - 6
    high: point_estimate + 6
    UI: "Estimated VO2max: ~{point_estimate} mL/kg/min (range {low}-{high})"
    Sources:
    Female: McArdle WD et al. Med Sci Sports 1972;4(4):182-186
    Male: Kline GM et al. Med Sci Sports Exerc 1987;19(3):253-259
    (YMCA step test male regression, same protocol)
    """
    if sex == "male":
        vo2 = 111.33 - (0.42 * recovery_hr)
    else:
        vo2 = 65.81 - (0.1847 * recovery_hr)
    return (round(vo2, 1), round(vo2-6, 1), round(vo2+6, 1))
    # indices: [0] = point_estimate, [1] = low, [2] = high

```

Validation: For age 18-25, `norm.ppf(0.05, 103, 12)` should be ~83 bpm and `norm.ppf(0.95, 103, 12)` should be ~123 bpm (reproducing YMCA Excellent/Poor bounds).

5.3b 6-Minute Walk Test — Cardiac-Condition Alternative

Administered to: Users with cardiac conditions (routed by application layer). **Method:** Percentage of predicted distance using sex-specific regression equations. **Source:** Enright PL, Sherrill DL. Am J Respir Crit Care Med 1998;158(5):1384-1387 (doi:10.1164/ajrccm.158.5.9710086). n=290 healthy adults aged 40-80.

Protocol: Flat, straight course of at least 10m (measured and marked by the user), 6 minutes self-paced walking back and forth. Record total distance walked in metres.

User-facing instruction: “Walk as far as you can in 6 minutes. Slow down or stop if needed, but cover as much ground as you can. If you feel dizzy, breathless, or feel like you are going to fall over, stop immediately.”

Course note: The original Enright & Sherrill reference equations used a 30m corridor. A 10m walkway introduces slightly more turning (~3–5% distance reduction vs. standard 30m course) but is acceptable. Treadmill or walk pad use is also permitted — belt assistance may produce slightly different distances than corridor walking, and the reference equations were not derived from treadmill data, so absolute percentiles may be marginally less precise. This is acceptable given the R^2 of the equations is ~40% regardless of surface. The primary value of repeat testing is within-person trend tracking, which remains valid on any consistent surface.

Safety note: 6MWT is self-paced and submaximal. It is the standard field test used in cardiac rehabilitation and is appropriate for users with stable cardiac conditions. It is NOT appropriate for users with unstable angina, recent MI (<1 month), or other acute cardiac events — the app should exclude such users from all exercise testing.

5.3b.1 Reference Equations (Enright & Sherrill 1998)

```
# normative/sixmwt.py
import numpy as np
from scipy.stats import norm as scipy_norm

def sixmwt_predicted(age: float, sex: str,
                    height_cm: float, weight_kg: float) -> float:
    """
    Predicted 6MWT distance in metres for a healthy reference individual.
    Enright & Sherrill 1998 equations are validated for ages 40-80.
    Age is clamped to [40, 80] – see note below.
    """
    age = float(np.clip(age, 40.0, 80.0)) # clamp: equations not validated outside 40-80
    if sex == "male":
        return (7.57 * height_cm) - (5.02 * age) - (1.76 * weight_kg) - 309.0
    else:
        return (2.11 * height_cm) - (2.29 * weight_kg) - (5.78 * age) + 667.0

def sixmwt_percentile(distance_m: float, age: float, sex: str,
                    height_cm: float, weight_kg: float) -> float:
    """
    Returns percentile 0-100 (population-relative, not percent-of-predicted).
    Method: z-score against the SD of the measured-distance distribution around
    the Enright & Sherrill predicted value for this individual.
    Enright & Sherrill report 95% CI of ±153m (men) / ±139m (women) around predicted.
    We model as normal with SD derived from published limits:
        men:    SD = 153 / 1.96 ≈ 78 m
        women:  SD = 139 / 1.96 ≈ 71 m
    Age clamping happens inside sixmwt_predicted.
    """
    predicted = sixmwt_predicted(age, sex, height_cm, weight_kg)
    sd = 78.0 if sex == "male" else 71.0
    z = (distance_m - predicted) / sd
    return float(scipy_norm.cdf(z) * 100)
```

Limitations: Enright & Sherrill equations were derived from adults aged 40-80. `sixmwt_predicted` clamps age to [40, 80] — for users outside this range the score is still computed but the UI should note reduced precision. R² of the equations is ~40%, reflecting that height/weight/age explain only part of variance.

Validation:

```
# Male, age 65, height 175 cm, weight 80 kg
pred = sixmwt_predicted(65, "male", 175, 80)
print(f"Predicted: {pred:.0f} m") # expected ~490-530 m range
pct = sixmwt_percentile(pred, 65, "male", 175, 80)
print(f"At predicted: {pct:.1f}") # expected ~50th percentile
```


5.4 Single-Leg Balance

Two protocols are administered depending on user age and condition status. The application layer routes to the correct metric_id before calling `compute_percentile`.

5.4.1 Standard Protocol — Eyes Open (age > 45, or any mobility-limiting, vestibular, or neurological condition)

Method: Truncated normal with ceiling at 60 s **Sources:** Springer BA et al. J Geriatr Phys Ther 2007;30(1): 8-15; Bohannon RW. Gait Posture 2018;63:99-105 (meta-analysis n=13,454)

Score input = average of left and right leg hold time (seconds). Cap each leg at 60 s before averaging.

Hand position: hands at sides throughout. Thigh of raised leg parallel to the floor.

Age Bucket	Mean (s)	SD (s)	Notes
18-39	45.0	14.0	Significant ceiling effect in healthy young adults
40-49	42.0	14.5	
50-59	38.0	15.0	
60-69	28.0	15.0	Springer et al. primary data
70-79	18.0	13.0	
80+	10.0	9.0	

```
# normative/balance.py
from scipy.stats import truncnorm
from normative.utils import balance_age_bucket

BALANCE_CEILING = 60.0
BALANCE_PARAMS = {
    (18, 39): (45.0, 14.0), (40, 49): (42.0, 14.5), (50, 59): (38.0, 15.0),
    (60, 69): (28.0, 15.0), (70, 79): (18.0, 13.0), (80, 99): (10.0, 9.0),
}

def balance_standard_percentile(hold_seconds: float, age: float) -> float:
    """metric_id: balance_standard"""
    hold_seconds = min(hold_seconds, BALANCE_CEILING)
    mu, sigma = BALANCE_PARAMS[balance_age_bucket(age)]
    a = (0 - mu) / sigma
    b = (BALANCE_CEILING - mu) / sigma
    dist = truncnorm(a, b, loc=mu, scale=sigma)
    return round(dist.cdf(hold_seconds) * 100, 1)
```

Ceiling handling: If `hold_seconds == 60.0`, display “Top ~X%” where `X = round(100 - balance_standard_percentile(59.9, age), 0)`, with note that true percentile is undetermined from this test.

5.4.2 Young/Healthy Protocol — Eyes Open + Eyes Closed (age ≤ 45, no mobility-limiting, vestibular, or neurological condition)

Rationale: Eyes-open single-leg balance has a severe ceiling effect in healthy young adults (mean ~45 s out of a 60 s max). Eyes-closed balance is substantially harder (typical healthy adult ~10-20 s) and provides better discrimination in this group.

Protocol: 1. Stand on right leg, left thigh raised parallel to the floor, hands at sides. Hold as long as possible, up to 60 seconds. 2. Rest for 30 seconds. 3. Stand on left leg, right thigh raised parallel to the floor, hands at sides. Hold as long as possible, up to 60 seconds. 4. Rest for 1 minute. 5. Repeat steps 1–3 with eyes closed. 6. Rest for 1 minute before beginning the next test.

Average right and left hold times separately for each condition, then compute composite score.

Scoring formula:

```
# normative/balance.py (continued)
def balance_young_percentile(eyes_open_s: float, eyes_closed_s: float) -> float:
    """
    metric_id: balance_young
    eyes_open_s: average of L+R eyes-open hold time (seconds, already averaged)
    eyes_closed_s: average of L+R eyes-closed hold time (seconds, already averaged)

    Score = ((eyes_open_s + eyes_closed_s) / 120) * 100

    Maximum = 100 (both conditions held for full 60 s each)
    This is a normalised raw-score metric, not a population percentile.
    Higher is always better.
    """
    eyes_open_s = min(eyes_open_s, 60.0)
    eyes_closed_s = min(eyes_closed_s, 60.0)
    return round(((eyes_open_s + eyes_closed_s) / 120.0) * 100, 1)
```

Note: `compute_percentile` receives `raw_score` as the **pre-averaged composite seconds** (`eyes_open_s + eyes_closed_s`). The division by 120 and multiplication by 100 happen inside the function. The application layer must pass the sum, not the average.

UI note: Display as “Balance Score: X / 100” rather than “Xth percentile” since this is a normalised score, not a population-referenced percentile.

Sources: - Eyes-open norms: Springer BA et al. J Geriatr Phys Ther 2007;30(1):8-15 - Eyes-closed reference: Bohannon RW et al. Arch Phys Med Rehabil 1984;65(12):762-764 (mean eyes-closed balance ~20 s in young healthy adults, declining with age)

5.5 Lower-Body Strength / Endurance

Two protocols are administered depending on user age and condition status.

5.5.1 Squat-to-Exhaustion Test (age ≤ 45, no mobility-limiting condition)

Rationale: The 30-second sit-to-stand test was designed as a functional test for older adults. In healthy young adults, scores are very high (mean ~33 reps), the task rewards rushing and tolerates poor form, and half-reps

are hard to detect. Bodyweight squats to exhaustion with enforced full form provide better discrimination and remove the pacing incentive.

Protocol: 1. Perform as many squats as possible. Spine must remain straight and each rep must reach appropriate depth (thighs parallel to the floor). 2. As soon as the user feels unable to perform another repetition, they stop the test and rest. Do not rest and resume. The test automatically concludes at the 2-minute mark.

Rep counting: The app counts reps automatically via the accelerometer. The user should be informed of the rep count in real time during the test.

WARNING — Preliminary normative base. Age-stratified squat-to-exhaustion norms for the general population are limited. The primary reference (Mayorga-Vega et al. 2015) used a pooled young-adult sample (mean age 28.5 ± 10 years, $n=381$). No age-stratified table exists. This metric is flagged preliminary and down-weighted in interpretation. Replace with better age-stratified data when available.

Source: Mayorga-Vega D et al. J Hum Kinet 2015;46:99-110 (doi:10.1515/hukin-2015-0038). Squats to exhaustion, $n=381$ (194M/187F), mean age 28.5 ± 10 yr.

Normative parameters (pooled young-adult, ~18-45 age range, verify against paper):

Sex	Mean (reps)	SD (reps)	Source
Male	38	12	Mayorga-Vega 2015 back-calc (verify)
Female	28	10	Mayorga-Vega 2015 back-calc (verify)

Note: These values are approximate back-calculations from percentile tables in Mayorga-Vega 2015. Claude Code should verify exact values against the paper's Table 2 before finalising. If paper values differ, use paper values.

```
# normative/sts.py – squat functions (add below STS_PARAMS and sts_percentile)
SQUAT_PARAMS = {
    "male": (38.0, 12.0),
    "female": (28.0, 10.0),
}

def squat_exhaustion_percentile(reps: float, sex: str) -> float:
    """
    metric_id: squat_exhaustion
    WARNING: Preliminary – single pooled age group, norms need verification.
    """
    mu, sigma = SQUAT_PARAMS[sex]
    return round(norm.cdf(reps, loc=mu, scale=sigma) * 100, 1)
```

UI: Display with disclaimer: “Preliminary benchmark — normative data for this test is limited to young adults. Your score is compared against a general young-adult reference population.”

5.5.2 30-Second Sit-to-Stand (age > 45 or mobility-limiting condition)

Method: Normal distribution, stitched from three sources **Sources:** - Ages 18-35: Lein DH et al. Int J Sports Phys Ther 2022;17(5):907-914 ($n=81$) - Ages 36-59: McKay MJ et al. 2017 (interpolated) — **Claude Code:**

locate full citation before implementing - Ages 60+: Rikli RE, Jones CJ. Gerontologist 1999;39(2):208-216 (updated 2013)

Age Bucket	Mean (reps)	SD (reps)	Source
18-35	33.0	5.4	Lein 2022
36-45	28.0	5.5	McKay 2017 interpolated
46-55	25.0	5.5	McKay 2017 interpolated
56-59	22.0	5.0	McKay 2017 interpolated
60-64	18.5	3.7	Rikli & Jones back-calc
65-69	17.0	3.7	Rikli & Jones back-calc
70-74	15.5	3.7	Rikli & Jones back-calc
75-79	14.0	3.7	Rikli & Jones back-calc
80+	12.0	3.7	Rikli & Jones back-calc

```
# normative/sts.py
from scipy.stats import norm
from normative.utils import sts_age_bucket

STS_PARAMS = {
    (18, 35): (33.0, 5.4), (36, 45): (28.0, 5.5), (46, 55): (25.0, 5.5),
    (56, 59): (22.0, 5.0), (60, 64): (18.5, 3.7), (65, 69): (17.0, 3.7),
    (70, 74): (15.5, 3.7), (75, 79): (14.0, 3.7), (80, 99): (12.0, 3.7),
}

def sts_percentile(reps: float, age: float) -> float:
    """metric_id: sts_30"""
    mu, sigma = STS_PARAMS[sts_age_bucket(age)]
    return round(norm.cdf(reps, loc=mu, scale=sigma) * 100, 1)

# Rep counting note: STS reps are counted automatically by the app accelerometer.
# The raw rep count is passed directly as reps.
```

5.6 Breath-Hold Duration

WARNING — Preliminary benchmark. Thin normative base (n=25). Down-weighted to 5% in Pulmonary Score (see Section 3.1). Display with disclaimer in UI.

Source: Alderson PO, Torchia MG. Radiology 1994;192(2):437-440 (n=25, mean=45 s, 95% CI 38-56 s => SD~9 s via back-calculation)

```
# normative/breath_hold.py
from scipy.stats import norm

BREATH_HOLD_MEAN = 45.0
BREATH_HOLD_SD = 9.0
```

```
def breath_hold_percentile(seconds: float) -> float:
    return round(norm.cdf(seconds, loc=BREATH_HOLD_MEAN,
                          scale=BREATH_HOLD_SD) * 100, 1)

# UI: show with text "Preliminary benchmark – limited reference population."
```

5.7 SpO2 Resting Oxygen Saturation

Method: Binary flag only — not a percentile, not weighted **Source:** O'Driscoll BR et al. Thorax 2017;72(Suppl 1):ii1-ii90 **Threshold:** $\geq 95\%$ = normal

```
# normative/spo2.py
def spo2_flag(spo2_pct: float) -> bool:
    """
    Returns True if SpO2 is NORMAL ( $\geq 95\%$ ) – i.e. True means OK, not True means problem.
    Named "flag" for historical consistency; pass directly to pulmonary_score() as spo2_ok.
    """
    return spo2_pct  $\geq$  95.0

# SpO2 < 95%: caps Pulmonary Score at 40 (see Section 3.1)
# Always display a warning banner when SpO2 < 95%, regardless of score.
```

5.8 I:E Ratio — Clinical Flag Only

Method: Binary flag — not a scored percentile metric. **Source:** General respiratory physiology; BTS/SIGN asthma guidelines. **Captured:** During resting baseline (continuous breath-by-breath gas analysis).

Normal resting I:E (inspiration:expiration) ratio is approximately 1:2. Deviations suggest pathology:

```
# normative/ie_ratio.py
IE_THRESHOLDS = {
    "reversed": 1.0, # I:E > 1:1 – strong flag (suggests air trapping / obstruction)
    "short_exp": 1/1.5, # I:E > 1:1.5 – warning (reduced expiratory time)
    "normal_min": 1/4.0, # I:E < 1:4 – prolonged expiration (check context)
}

def ie_ratio_flag(inspiration_s: float, expiration_s: float) -> str:
    """
    Returns: 'reversed', 'short_expiration', 'prolonged_expiration', or 'normal'
    This flag feeds Appendix C clinical flags, not compute_percentile.
    """
    ratio = inspiration_s / expiration_s
    if ratio  $\geq$  IE_THRESHOLDS["reversed"]:
        return "reversed"
    elif ratio  $\geq$  IE_THRESHOLDS["short_exp"]:
        return "short_expiration"
    elif ratio < IE_THRESHOLDS["normal_min"]:
        return "prolonged_expiration"
    return "normal"
```

UI: Display warning banner for “reversed” or “short_expiration” flags. “Prolonged expiration” may be normal in some users (slower breathers) — display informational note only, not a warning.

5.9 HRV — RMSSD

Method: Log-normal distribution. RMSSD is right-skewed; log-transform before scoring. **Source:** Nunan DA et al. Int J Cardiol 2010;144(3):381-385 (systematic review, age-stratified short-term HRV norms in healthy adults). Supplemented by Shaffer F, Ginsberg JP. Front Public Health 2017;5:258 (doi:10.3389/fpubh.2017.00258). **Captured:** 20/30-min resting baseline only (4-min window including controlled breathing). **Not inverted:** higher RMSSD = better autonomic function.

RMSSD follows a log-normal distribution. We fit $\ln(\text{RMSSD})$ as a normal and compute percentile in log-space. Log SD is approximately 0.51 across all age/sex groups (CV ~55%, stable across age — Nunan 2010).

Normative parameters (geometric means by age — verify against Nunan 2010 Table 3):

Age Bucket	Geo. Mean RMSSD (ms)	$\ln(\text{geo. mean})$	$\ln \text{SD}$
18-29	42	3.74	0.51
30-39	35	3.56	0.51
40-49	29	3.37	0.51
50-59	25	3.22	0.51
60-69	21	3.04	0.51
70+	18	2.89	0.51

Note: Nunan 2010 does not stratify by sex. Sex differences in RMSSD are modest at rest (females slightly higher) and not consistently reported across studies. Use pooled norms for v1; add sex stratification when internal data permits.

Note: These values are approximate. Claude Code must verify exact geometric means against Nunan 2010 Table 3 before finalising.

```
# normative/hrv.py
import numpy as np
from scipy.stats import norm
from normative.utils import hrv_age_bucket

HRV_LOG_SD = 0.51 # approximately constant across age groups (Nunan 2010)

HRV_PARAMS = {
    # (age_bucket): ln(geometric_mean) - verify against Nunan 2010 Table 3
    (18, 29): 3.74,
    (30, 39): 3.56,
    (40, 49): 3.37,
    (50, 59): 3.22,
    (60, 69): 3.04,
    (70, 99): 2.89,
}
```

```
def hrv_percentile(rmssd_ms: float, age: float, sex: str = None) -> float:
    """
    metric_id: hrv_rmssd
    sex: accepted for API consistency but not used in v1 (pooled norms).
    rmssd_ms: RMSSD in milliseconds (raw value from device).
    """
    if rmssd_ms <= 0:
        return 0.0
    log_mean = HRV_PARAMS[hrv_age_bucket(age)]
    z = (np.log(rmssd_ms) - log_mean) / HRV_LOG_SD
    return round(float(norm.cdf(z) * 100), 1)
```

Validation:

```
# Age 30, RMSSD = 35ms (at geometric mean) → should be ~50th percentile
pct = hrv_percentile(35.0, 30)
print(f"At geo mean: {pct:.1f}") # expected ~50th

# Age 30, RMSSD = 20ms (below mean) → should be ~20-30th percentile
pct_low = hrv_percentile(20.0, 30)
print(f"Low RMSSD: {pct_low:.1f}")
```

5.10 Front Plank — Core Endurance

Method: Truncated normal with ceiling at 180 s (3-min cap). Sex-stratified. **Sources:** McGill SM et al. Clin Biomech 1999;14(6):389-398 (endurance test battery, foundational plank protocol); Strand SL et al. J Strength Cond Res 2014;28(10):2972-2981 (doi:10.1519/JSC.0000000000000493) (age/sex stratified norms, n=614).

Protocol: Forearm plank, elbows under shoulders, body straight head to heels, toes on floor. Hold until form breakdown (hip sag detected by accelerometer, or self-reported). 3-minute hard cap. **Version:** 30-min only.

WARNING — verify normative values. The parameters below are approximate back-calculations from Strand et al. 2014. Claude Code must verify against the paper's normative tables before finalising. If paper values differ, use paper values.

Sex	Age Bucket	Mean (s)	SD (s)	Source
Male	18-39	70	25	Strand 2014 (verify)
Male	40-59	50	22	Strand 2014 (verify)
Male	60+	30	18	Strand 2014 (verify)
Female	18-39	60	22	Strand 2014 (verify)
Female	40-59	42	20	Strand 2014 (verify)
Female	60+	25	15	Strand 2014 (verify)

```
# normative/plank.py
from scipy.stats import truncnorm
from normative.utils import plank_age_bucket
```

```

PLANK_CEILING = 180.0 # 3-minute cap

PLANK_PARAMS = {
    # (sex, age_bucket): (mean_s, sd_s) – verify against Strand 2014
    ("male", (18,39)): (70.0, 25.0),
    ("male", (40,59)): (50.0, 22.0),
    ("male", (60,99)): (30.0, 18.0),
    ("female", (18,39)): (60.0, 22.0),
    ("female", (40,59)): (42.0, 20.0),
    ("female", (60,99)): (25.0, 15.0),
}

def plank_percentile(hold_seconds: float, age: float, sex: str) -> float:
    """metric_id: plank_duration"""
    hold_seconds = min(hold_seconds, PLANK_CEILING)
    mu, sigma = PLANK_PARAMS[(sex, plank_age_bucket(age))]
    a = (0 - mu) / sigma
    b = (PLANK_CEILING - mu) / sigma
    dist = truncnorm(a, b, loc=mu, scale=sigma)
    return round(dist.cdf(hold_seconds) * 100, 1)

```

Ceiling handling: If hold_seconds == 180.0, display “Top ~X%” consistent with balance ceiling handling (Section 5.4.1).

5.11 Wall Sit — Isometric Quad Endurance

WARNING — Preliminary normative base. Age/sex-stratified wall sit norms are limited in the literature. Values below are approximate estimates from general fitness testing literature. Replace with better data when available.

Method: Truncated normal with ceiling at 120 s (2-min cap). Sex-stratified. **Protocol:** Back flat against wall, hips and knees at 90°. App verifies knee angle via tilt sensor continuously. Hold until unable to maintain 90° position. 2-minute cap. **Version:** 30-min only.

Sex	Age Bucket	Mean (s)	SD (s)	Source
Male	18-39	85	30	General fitness literature (preliminary)
Male	40-59	60	25	General fitness literature (preliminary)
Male	60+	35	20	General fitness literature (preliminary)
Female	18-39	75	28	General fitness literature (preliminary)
Female	40-59	52	22	General fitness literature (preliminary)
Female	60+	28	18	General fitness literature (preliminary)

```

# normative/wall_sit.py
from scipy.stats import truncnorm
from normative.utils import wall_sit_age_bucket

WALL_SIT_CEILING = 120.0 # 2-minute cap

```



```

WALL_SIT_PARAMS = {
    # (sex, age_bucket): (mean_s, sd_s) – PRELIMINARY, replace when better data available
    ("male", (18,39)): (85.0, 30.0),
    ("male", (40,59)): (60.0, 25.0),
    ("male", (60,99)): (35.0, 20.0),
    ("female", (18,39)): (75.0, 28.0),
    ("female", (40,59)): (52.0, 22.0),
    ("female", (60,99)): (28.0, 18.0),
}

def wall_sit_percentile(hold_seconds: float, age: float, sex: str) -> float:
    """
    metric_id: wall_sit_duration
    WARNING: Preliminary – normative base is thin. Replace when peer-reviewed
    age/sex-stratified data becomes available.
    """
    hold_seconds = min(hold_seconds, WALL_SIT_CEILING)
    mu, sigma = WALL_SIT_PARAMS[(sex, wall_sit_age_bucket(age))]
    a = (0 - mu) / sigma
    b = (WALL_SIT_CEILING - mu) / sigma
    dist = truncnorm(a, b, loc=mu, scale=sigma)
    return round(dist.cdf(hold_seconds) * 100, 1)

```

UI: Display with disclaimer: “Preliminary benchmark — normative data for this test is limited. Score reflects estimated population norms pending peer-reviewed age/sex-stratified reference data.”

6. OFFLINE SETUP TASKS (Run Once Before First Deployment)

Task 1 — Download and Fit NHANES KDE

```

pip install pandas scipy pyreadstat
mkdir nhanes_raw && cd nhanes_raw
# Download all 32 XPT files from URLs in Section 5.2.1
# (8 cycles x 4 files: BPX, DEMO, MCQ, BPQ)
cd ..
python -c "from normative.nhanes import build_nhanes_kde; build_nhanes_kde()"
# Output: normative/data/nhanes_hr_kde.pkl
# Commit this .pkl file to the repo – do NOT re-download at runtime

```

Task 2 — Commit GLI Lookup Table

The GLI lookup table is pre-populated by calling the ERS GLI API offline. The current CSV (`normative/data/gli_lookup.csv`) has 900 adult rows covering ages 19–84 across 9 age points and 16 height values per sex/ethnicity group. Commit this file to the repo.

```

cp gli_lookup.csv normative/data/gli_lookup.csv
git add normative/data/gli_lookup.csv
git commit -m "Add GLI lookup table (900 adult rows, v1)"

```

Expanding coverage over time: As additional API quota becomes available (1,000 calls/month), add rows to fill in missing ages. Priority order: 1. Fill age gaps in 30–70 range (currently 41 → 52 is the largest gap) 2. Dense 1-year coverage for ages 18–85 3. Add sex=m heights above 188.0 cm if needed

When adding rows, re-run Task 3 validation to confirm interpolation accuracy has improved.

Task 3 — Validate GLI Lookup Interpolation

```
# On-grid point – result should match raw CSV value exactly
pct = gli_percentile(4.063, 30, "male", 172.7, "fev1", "ne_asian")
print(f"On-grid: {pct:.1f}") # expected ~50th (measured == M)

# Off-grid interpolation – age 27 sits between age-24 and age-30 grid points
pct_27 = gli_percentile(4.0, 27, "male", 172.7, "fev1", "caucasian")
pct_24 = gli_percentile(4.0, 24, "male", 172.7, "fev1", "caucasian")
pct_30 = gli_percentile(4.0, 30, "male", 172.7, "fev1", "caucasian")
print(f"Age 24: {pct_24:.1f}, Age 27 (interp): {pct_27:.1f}, Age 30: {pct_30:.1f}")
# pct_27 should be between pct_24 and pct_30

# Ethnicity check – african_american should yield higher percentile than caucasian
# for the same absolute measurement (lower predicted M → same value looks better)
pct_cau = gli_percentile(3.5, 30, "male", 172.7, "fev1", "caucasian")
pct_aa = gli_percentile(3.5, 30, "male", 172.7, "fev1", "african_american")
print(f"Caucasian: {pct_cau:.1f}, African American: {pct_aa:.1f}")
assert pct_aa > pct_cau, "African American should have higher percentile for same measurement"
```

Task 4 — Validate YMCA Parameters

```
# Should reproduce YMCA category thresholds
from scipy.stats import norm
mean, sd = 103, 12 # age 18-25
print(f"5th pct: {norm.ppf(0.05, mean, sd):.1f}") # expected ~83 bpm
print(f"95th pct: {norm.ppf(0.95, mean, sd):.1f}") # expected ~123 bpm
```

7. RUNTIME MODULE INITIALIZATION

```
# normative/gli.py – module-level globals (loaded on first import)
# See Section 5.1.3 for the full loading and indexing code.
# Summary: reads gli_lookup.csv, drops null rows, pre-indexes by (sex, ethnicity).

# ETHNICITY_CODE dict and gli_percentile() are also defined here (see Section 5.1)

# _____
# normative/__init__.py – loaded once at app startup
# All data loading happens inside each submodule (gli.py, nhanes.py) to avoid circular imports

from normative.gli import gli_percentile
from normative.nhanes import resting_hr_percentile # KDE_DICT loaded inside nhanes.py
from normative.ymca import ymca_hr_percentile, estimate_vo2max
from normative.sixmwt import sixmwt_percentile, sixmwt_predicted
from normative.balance import balance_standard_percentile, balance_young_percentile
```

```

from normative.sts import sts_percentile, squat_exhaustion_percentile
from normative.hrv import hrv_percentile
from normative.plank import plank_percentile
from normative.wall_sit import wall_sit_percentile
from normative.breath_hold import breath_hold_percentile
from normative.spo2 import spo2_flag
from normative.ie_ratio import ie_ratio_flag
from normative.scoring import pulmonary_score, cardiovascular_score, mobility_score

```

8. ACADEMIC REFERENCES

Component	Citation
GLI-2012 Spirometry	Quanjer PH et al. Eur Respir J 2012;40(6):1324-1343
GLI API (lookup table source)	gli-calculator.ersnet.org — ERS GLI Tools (implements Quanjer 2012)
NHANES Resting HR	CDC/NCHS. NHANES 2003-2018. wwwn.cdc.gov/nchs/nhanes
YMCA Step Test	Golding LA et al. Y's Way to Physical Fitness. YMCA, 1989
VO2max Regression (female)	McArdle WD et al. Med Sci Sports 1972;4(4):182-186
VO2max Regression (male)	Kline GM et al. Med Sci Sports Exerc 1987;19(3):253-259
FRIEND VO2max norms	Kaminsky LA et al. Mayo Clin Proc 2015;90(11):1515-1523 (reserved — not currently implemented; would support VO2max percentile display if that feature is added)
Single-Leg Balance	Springer BA et al. J Geriatr Phys Ther 2007;30(1):8-15
Balance meta-analysis	Bohannon RW. Gait Posture 2018;63:99-105
STS young adults	Lein DH et al. Int J Sports Phys Ther 2022;17(5):907-914
STS older adults	Rikli RE, Jones CJ. Gerontologist 1999;39(2):208-216
Squat-to-exhaustion norms	Mayorga-Vega D et al. J Hum Kinet 2015;46:99-110
6MWT reference equations	Enright PL, Sherrill DL. Am J Respir Crit Care Med 1998;158(5):1384-1387
6MWT ATS guidelines	ATS Committee. Am J Respir Crit Care Med 2002;166(1):111-117
Balance eyes-closed	Bohannon RW et al. Arch Phys Med Rehabil 1984;65(12):762-764
HRV RMSSD norms	Nunan DA et al. Int J Cardiol 2010;144(3):381-385
HRV review	Shaffer F, Ginsberg JP. Front Public Health 2017;5:258
Front Plank protocol	McGill SM et al. Clin Biomech 1999;14(6):389-398

Component	Citation
Front Plank norms	Strand SL et al. J Strength Cond Res 2014;28(10): 2972-2981
I:E ratio / respiratory physiology	BTS/SIGN Asthma Guidelines (current edition)
Breath-Hold	Alderson PO, Torchia MG. Radiology 1994;192(2):437-440
SpO2 threshold	O'Driscoll BR et al. Thorax 2017;72(Suppl 1):ii1-ii90

9. DESIGN DECISIONS SUMMARY

Decision	Rationale
No single aggregate score	Prevents masking within-domain weaknesses
Decade age buckets, no interpolation	Avoids false precision given reference SD
Sex stratification varies by metric	Sex-stratified: GLI spirometry, 6MWT (Enright & Sherrill), squat (SQUAT_PARAMS), plank (PLANK_PARAMS), wall sit (WALL_SIT_PARAMS). Sex-pooled: NHANES resting HR, YMCA step test, balance, STS, HRV v1. Pooled metrics have weaker or inconsistent sex-stratified reference data; revisit as internal data accumulates.
GLI-2012 with ethnicity	Better per-individual accuracy; d offset shifts predicted median up to ~12%
Store raw scores only	Enables recomputation when references update
Balance ceiling -> "Top X%"	Distribution censored in young adults
Breath-hold 5% weight	Weakest normative base (n=25); HRV added at 10%, reducing breath-hold share
NHANES KDE over parametric	Resting HR is non-normal
VO2max display-only	SEE ~6 mL/kg/min too wide for scoring
Squat test preliminary flag	Only pooled young-adult norms exist (Mayorga-Vega 2015); no age stratification
6MWT for cardiac users	Safest validated field test for stable cardiac conditions; self-paced and submaximal
6MWT labelled "Functional Exercise Capacity"	6MWT measures integrated functional capacity, not peak cardiovascular fitness
Balance eyes-closed for age ≤ 45	Eyes-open balance has ceiling effect (mean ~45s vs 60s max) in healthy young adults
Routing in app layer, not normative engine	compute_percentile is stateless; protocol selection based on user profile conditions

Decision	Rationale
Version-conditional weight normalisation	Cleaner than hard-coding separate score functions per version; preserves 0-100 scale across versions
HRV pooled by sex (v1)	Sex differences in resting RMSSD are modest and inconsistently reported; revisit with internal data
Wall sit flagged preliminary	No sufficiently powered peer-reviewed age/sex-stratified normative dataset currently identified
I:E ratio flag-only	No population normative dataset supports scored percentiles; clinical thresholds are well-established
Plank/wall sit in Functional Mobility	Core and isometric quad endurance are musculoskeletal constructs; Cardiovascular score already has two metrics
balance_young treated as equivalent to percentile in mobility_score()	No adequate reference population exists for the composite eyes-open + eyes-closed protocol. The 0–100 normalised score is designed to be interpretively comparable (50 = middling, 100 = perfect hold on both legs both conditions). The UI should flag this distinction for young-balance users rather than presenting the mobility score as purely population-referenced.
resting_hr and hrv_rmssd in Pulmonary Score (not Cardiovascular)	These reflect autonomic regulation of the cardiorespiratory system at rest. The Cardiovascular Score captures <i>exercise response</i> (step test or 6MWT); resting HR and HRV complement the resting spirometry and SpO ₂ measurements as baseline cardiorespiratory health markers. Placing them here avoids making the Cardiovascular Score redundant with the resting baseline. UI domain label should read “Cardiorespiratory” or similar if this placement is surfaced to users.

APPENDIX A: compute_percentile() Dispatch Body

```
# normative/__init__.py (continued – add after module loading)

def compute_percentile(metric_id: str, raw_score: float, age: float,
                      sex: str = None, height_cm: float = None,
                      weight_kg: float = None,
                      ethnicity: str = "caucasian") -> float:
    """
    Unified entry point. Returns percentile 0-100 (higher = always better).
    Inverted metrics (resting_hr, ymca_recovery_hr) are already flipped inside
    their respective functions.

    ethnicity: required for fev1/fvc/fev1_fvc.
    weight_kg: required for sixmwt_distance.
    sex:       required for fev1/fvc/fev1_fvc, sixmwt_distance, squat_exhaustion.
```

```

For balance_young, raw_score = eyes_open_s + eyes_closed_s (sum, not average).
"""
if metric_id == "resting_hr":
    return resting_hr_percentile(raw_score, age)

elif metric_id == "breath_hold":
    return breath_hold_percentile(raw_score)

elif metric_id == "fev1":
    return gli_percentile(raw_score, age, sex, height_cm, "fev1", ethnicity)

elif metric_id == "fvc":
    return gli_percentile(raw_score, age, sex, height_cm, "fvc", ethnicity)

elif metric_id == "fev1_fvc":
    return gli_percentile(raw_score, age, sex, height_cm, "fev1fvc", ethnicity)

elif metric_id == "ymca_recovery_hr":
    return ymca_hr_percentile(raw_score, age)

elif metric_id == "sixmwt_distance":
    return sixmwt_percentile(raw_score, age, sex, height_cm, weight_kg)

elif metric_id == "balance_standard":
    return balance_standard_percentile(raw_score, age)

elif metric_id == "balance_young":
    # raw_score = eyes_open_s + eyes_closed_s (combined seconds, max 120)
    # balance_young_percentile is best called directly by the app with both values;
    # for consistency via this dispatcher, pass the sum and let the function divide by 120.

    return round((min(raw_score, 120.0) / 120.0) * 100, 1)

elif metric_id == "sts_30":
    return sts_percentile(raw_score, age)

elif metric_id == "squat_exhaustion":
    return squat_exhaustion_percentile(raw_score, sex)

elif metric_id == "hrv_rmssd":
    return hrv_percentile(raw_score, age, sex)

elif metric_id == "plank_duration":
    return plank_percentile(raw_score, age, sex)

elif metric_id == "wall_sit_duration":
    return wall_sit_percentile(raw_score, age, sex)

else:
    raise ValueError(f"Unknown metric_id: {metric_id!r}")

```

Note on `breath_hold` and `age` : The `age` parameter is accepted by `compute_percentile` for API consistency but is not forwarded to `breath_hold_percentile` because no age-stratified normative data exists — the single reference study (Alderson & Torchia 1994) provides one pooled estimate only.

Note on `balance_young` : The app layer should call `balance_young_percentile(eyes_open_s, eyes_closed_s)` directly rather than routing through `compute_percentile`, since the metric requires two input values. The dispatcher supports a single `raw_score` fallback (sum of both times) for pipeline consistency.

Note on `sixmwt_distance` : Enright & Sherrill equations are validated for ages 40-80. Age clamping to [40, 80] is implemented inside `sixmwt_predicted` directly. For cardiac patients outside this range the UI should note that reference values may be less precise.

APPENDIX B: Recommended Module File Layout

```
normative/
  __init__.py      ← compute_percentile() dispatch + imports from all submodules (KDE_DICT loaded
in nhanes.py, GLI_LOOKUP loaded in gli.py)
  utils.py         ← all _bucket() / age_bucket helpers
  gli.py           ← gli_percentile(), _bilinear_lms(), GLI_LOOKUP, ETHNICITY_CODE (all GLI
globals here to avoid circular import)
  nhanes.py        ← build_nhanes_kde() (offline script), KDE_DICT (loaded at import),
resting_hr_percentile()
  ymca.py          ← ymca_hr_percentile(), estimate_vo2max(recovery_hr, sex), YMCA_PARAMS
  sixmwt.py        ← sixmwt_predicted(), sixmwt_percentile()
  balance.py       ← balance_standard_percentile(), balance_young_percentile(), BALANCE_PARAMS
  sts.py           ← sts_percentile(), squat_exhaustion_percentile(), STS_PARAMS, SQUAT_PARAMS
  hrv.py           ← hrv_percentile(), HRV_PARAMS (20/30-min only)
  plank.py         ← plank_percentile(), PLANK_PARAMS (30-min only)
  wall_sit.py      ← wall_sit_percentile(), WALL_SIT_PARAMS (30-min only)
  breath_hold.py   ← breath_hold_percentile(), BREATH_HOLD_MEAN/SD
  spo2.py          ← spo2_flag()
  ie_ratio.py      ← ie_ratio_flag(), IE_THRESHOLDS
  scoring.py       ← pulmonary_score(), cardiovascular_score(), mobility_score()
  data/
    gli_lookup.csv  ← committed in Task 2 (pre-populated via ERS GLI API)
    nhanes_hr_kde.pkl ← generated in Task 1
```

APPENDIX C: Clinical Flags Reference

Clinical flags are signals that warrant follow-up, contextualisation, or referral. They are distinct from percentile scores — a user can score well on a test and still trigger a flag (e.g. reversed I:E ratio despite normal spirometry percentile). Flags do not constitute a diagnosis.

****Flags implemented in this engine (normative/*):****

Flag	Module	Threshold	Source
SpO ₂ low	spo2.py	< 95% → caps Pulmonary Score at 40	O'Driscoll et al. Thorax 2017
I:E reversed	ie_ratio.py	I:E > 1:1 → “reversed” warning	BTS/SIGN; general resp. physiology
I:E short expiration	ie_ratio.py		BTS/SIGN

Flag	Module	Threshold	Source
		I:E > 1:1.5 → "short_expiration" warning	
I:E prolonged expiration	ie_ratio.py	I:E < 1:4 → informational note	BTS/SIGN

Flags implemented in the application layer (not this engine):

The following flag categories require clinical thresholds and cross-test logic to be defined by the clinical team and implemented in the app layer. The normative engine supplies the raw percentile inputs; flag logic lives outside it.

Category	Inputs from engine	TODO
A1 — Cardiovascular	resting_hr percentile, ymca/6mwt percentile, hrv_rmssd percentile	Define thresholds (e.g. resting HR > 100 bpm, HRV < 10th pct)
A2 — Pulmonary	fev1/fvc/fev1_fvc percentiles, SpO ₂ flag, I:E flag	Define spirometry z-score cutoffs (e.g. FEV1 < 5th pct = flag)
A3 — Functional Mobility	balance, STS/squat, plank, wall sit percentiles	Define low-score thresholds per domain
A4 — Composite / Cross-test	All percentile outputs	Define cross-test patterns (e.g. low FEV1 + low 6MWT + low HRV)
A5 — Absolute Stop Conditions	n/a — app-side only	Dizziness, chest pain, SpO ₂ < 88%, self-report; halt test immediately

A6 — Condition Routing: The conditions that determine cardiac-routing (→ 6MWT instead of YMCA) and mobility-routing (→ 30-sec STS instead of squat) are stored in the user profile in the app database. Claude Code should refer to the app's conditions schema to identify which condition codes qualify as cardiac-limiting or mobility-limiting. The normative engine is stateless — it receives only the metric_id already determined by the routing layer.